Generative AI

Project Proposal

Submitted to: Mam Shahela Saif

Submitted By:
Umar Farooq 21i-1143
Ibrahim Ahmed 22i-8781

Dated: October 03, 2025

Track 1 Research Paper Proposal – AI-Text Detection

Title

Detecting AI-Generated vs. Human-Written Text Using Transformer-Based Models

Abstract

The rise of advanced Large Language Models (LLMs), such as GPT-4, has made AI-generated text nearly indistinguishable from human-written text. This poses challenges in education, journalism, and research integrity. This project proposes building a text classifier that distinguishes between human-written and AI-generated text using publicly available datasets. The scope is restricted to English text for feasibility. The goal is to benchmark existing methods and evaluate robustness against paraphrased AI text.

Problem Statement

Existing AI-text detectors are unreliable, often failing when text is slightly rephrased or paraphrased. A lightweight yet effective detection system is needed to support academic and professional environments where text authenticity is critical.

Objectives

- Train and compare **2–3 classifiers** (e.g., Logistic Regression with stylometry features, BERT, RoBERTa).
- Evaluate performance using standard metrics (**accuracy, precision, recall, F1**).
- Test robustness against paraphrased AI-generated text.

Dataset

- **HC3 Dataset** (Human vs. ChatGPT responses).
- **Kaggle AI vs. Human Text Classification dataset**.
- Both datasets are openly available and suitable for this project.

Methodology

1. **Preprocessing:** Clean, tokenize, and normalize text.
2. **Model Training:**
    - Logistic Regression (baseline).
    - BERT-based classifier.
    - RoBERTa classifier.
3. **Evaluation:** Standard metrics + adversarial test (paraphrased samples).
4. **Analysis:** Highlight strengths, weaknesses, and practical applicability.

Expected Outcomes

- A benchmark of detection accuracy for multiple models.
- Insights into robustness against paraphrased AI text.
- A lightweight prototype detector (CLI or notebook-based).

Feasibility (2-Month Scope)

- Dataset: Readily available.
- Models: Pre-trained, limited fine-tuning required.
- Tools: Python, HuggingFace Transformers, Scikit-learn.
- Timeline:
    - Week 1–2: Dataset prep, baseline setup.
    - Week 3–5: Model training + evaluation.
    - Week 6–7: Robustness testing + analysis.
    - Week 8: Report writing + submission.

Track 2 Marketable Product Proposal – StudyGenAI: AI-Powered Study Assistant

Title

StudyGenAI – An AI-Powered Study Assistant for University Students

Abstract

Students often struggle to process large amounts of study material under time constraints. StudyGenAI is a lightweight web application that helps university students learn more effectively by leveraging Generative AI. Users can upload or paste their study material, and the system will summarize content, generate quiz questions, and provide simplified explanations. The goal is to create a practical, usable tool within the semester timeframe.

Problem Statement

Traditional study methods are time-consuming and inefficient. Existing tools like Quizlet and Grammarly do not fully leverage LLMs for summarization, question generation, and concept explanation in one unified platform.

Objectives

- Build a web app that allows students to:
    - Upload/paste study material.
    - Generate **summaries** of the content.

    - Create **quiz questions** (MCQs/short answer).
    - Provide **simplified explanations** (“Explain like I’m 5”).
- Ensure clean and simple UI for accessibility.
- Enable export of generated content (PDF or text).

Dataset & Models

- Input comes directly from student-provided material (PDF/text).
- Models: OpenAI GPT API (or HuggingFace models like LLaMA-2/3).
- Libraries: LangChain, PyPDF2 for PDF handling.

Methodology

1. **Frontend:** React + TailwindCSS.
2. **Backend:** Node.js or Python (FastAPI).
3. **Core Features:**
    - Text/PDF input.
    - Summarization via LLM.
    - Quiz generation via LLM prompt.
    - Simplified explanations.
    - Export as PDF/text.
4. **Testing:** User testing with classmates to validate usefulness.

Expected Outcomes

- A working demo web app with core study assistant features.
- Evidence of usability and efficiency for student learning.
- Documentation of technical architecture and implementation.

Feasibility (2-Month Scope)

- Focus on **single-document processing only** (no large-scale retrieval).
- API-driven LLM use (no training required).
- Timeline:
    - Week 1–2: Project setup + UI/UX scaffolding.
    - Week 3–5: Integrate summarization + quiz + explanation.
    - Week 6–7: Testing + export feature.
    - Week 8: Final polish + submission.
